

Julia crash course - Vietnam

How to work

- Create the files named in each section inside one folder called JuliaBootcamp.
- Run your code often. Do not write a whole section before testing it.
- Use comments like # Section 2 or # My answer to label your work.
- When an error appears, read the first error message before asking for help.

Initial worksheet: first code in VS Code

Do this part while the instructor helps with setup problems. It is written step by step because we have not introduced the Julia syntax yet.

Step 1. Create your first Julia file

- Open VS Code.
- Create a folder called JuliaBootcamp.
- Inside that folder, create a file called 00_first_steps.jl.
- Save the file before running it.

Step 2. Make Julia print something

Write this in 00_first_steps.jl and run the file:

```
println("Hello Julia")
```

You should see the text printed in the output or terminal. Then change the text so it prints your own name or group name.

Step 3. Print simple calculations

Add the following lines below your first line and run the file again:

```
println(2 + 2)
println(100 * 1.05)
```

Now add one more printed calculation of your own.

Step 4. Create variables and print them

Add this code. Run the file after adding it:

```
country = "Vietnam"
println(country)

gdp = 1000.0
growth = 0.08
gdp_next_year = gdp * (1 + growth)
println(gdp_next_year)
```

Then add variables called exports and imports. Use them to calculate and print resource_balance:

```
resource_balance = gdp + exports - imports
println(resource_balance)
```

You must choose the values for exports and imports yourself before this code will run.

Step 5. Make one error and fix it

Copy this line into your file and run it:

```
println(gdpp)
```

Read the error message. Then fix the line so it prints the GDP variable you created earlier.

1. Pkg, projects, and packages

Create a new file called `01_pkg_environment.jl`. In this exercise you will create a small Julia project, add a package, and check that Julia records the package in the project files.

A package is code someone else has written that we can use in our own project. A project environment is the list of packages used by one project folder.

Start from this code

```
using Pkg

Pkg.activate(".")
Pkg.add("Example")

using Example
println(hello("Julia"))
```

Task 1: create and run the project

- Create a folder called JuliaBootcamp.
- Inside that folder, create a file called `01_pkg_environment.jl`.
- Copy the starter code into the file.
- Run the file.
- Check that the output prints a hello message from the Example package.

Task 2: inspect what Julia created

- Look inside your JuliaBootcamp folder after the file has run.
- Find the file called `Project.toml`.
- Find the file called `Manifest.toml`.
- Open `Project.toml`.
- Find the line where `Example` is listed as a dependency of the project.

Task 3: use the same package from a second file

- Create a second file in the same JuliaBootcamp folder called `01_test_package.jl`.
- Write this code in the second file:

```
using Pkg

Pkg.activate(".")

using Example
println(hello("Model class"))
```

- Run the second file.
- Do not write `Pkg.add("Example")` in the second file. The package was already added to the project.
- Open `Project.toml` again and check that `Example` is still listed.

2. Variables and basic objects

Create a new file called `02_variables.jl`. Start by writing and running the code below.

Start from this code

```
base_year = 2024
horizon = 5
country = "Vietnam"
gdp = 1000.0
```

```

growth = 0.08
variable_name = :GDP

gdp_next = gdp * (1 + growth)
println(gdp_next)

println(typeof(country))
println(typeof(variable_name))

```

Add more variables

- Add variables called exports and imports.
- Choose your own numeric values for exports and imports.
- Calculate resource_balance using `gdp + exports - imports`.
- Print resource_balance.

Inspect object types

- Print `typeof(base_year)`.
- Print `typeof(gdp)`.
- Print `typeof(country)`.
- Print `typeof(variable_name)`.

Create and fix a capitalization error

Add this line and run the file:

```
println(GDP)
```

Julia treats GDP and gdp as different names. Read the error, then change the line so it prints the existing gdp variable.

3. Vectors, ranges, and indexing

Create a new file called `03_vectors_indexing.jl`. This section is about lists of values and selecting elements from them.

Start from this code

```

years = 2024:2029
sectors = [:agriculture, :industry, :services]

println(first(years))
println(last(years))
println(sectors[1])
println(sectors[end])

values = [100.0, 110.0, 121.0]
println(values[1])
println(values[end])

```

Work with years and sectors

- Change the years range so it starts in 2025 and ends in 2030.
- Add a fourth sector to sectors, for example `:energy`.
- Print the first sector and the last sector again.

Work with a vector of GDP values

- Create a vector called `gdp_values` with at least five numbers.
- Print the first value in `gdp_values`.
- Print the last value in `gdp_values`.
- Calculate `gdp_change` as the last value minus the first value, then print it.

Create a growth path

Add and run this code:

```
path = 100 .* (1 + 0.05) .^ (1:5)
println(path)
```

- Change the starting value from 100 to 200.
- Change the growth rate from 0.05 to 0.08.
- Rerun the file and compare the printed path.

4. Dictionaries and named tuples

Create a new file called 04_dictionaries_named_tuples.jl. Dictionaries are useful when you want to store values by name.

Start from this code

```
gdp = Dict{
    :agriculture => 100.0,
    :industry   => 300.0,
    :services   => 600.0,
}

println(gdp[:industry])

total_gdp = gdp[:agriculture] + gdp[:industry] + gdp[:services]
println(total_gdp)

settings = (
    base_year = 2024,
    horizon   = 5,
    country   = "Vietnam",
)

println(settings.base_year)
println(settings.country)
```

Extend the GDP dictionary

- Change at least one GDP value and rerun the file.
- Add a fourth sector called :energy with its own GDP value.
- Update total_gdp so it includes the new sector.
- Print the new total_gdp.

Create an exports dictionary

- Create a dictionary called exports with one value for each sector.
- Calculate total_exports by adding the values from exports.
- Print total_exports.

Use named tuple fields

- Add a field called model_name to settings.
- Print settings.model_name.

Then intentionally run this line:

```
println(gdp[:Industry])
```

Read the error, then change it back to the correct key.

5. Functions

Create a new file called 05_functions.jl. Functions let you reuse the same calculation with different inputs.

Start from this code

```
function grow(value, growth_rate)
    return value * (1 + growth_rate)
end

println(grow(100, 0.05))

function project_one_year(value; growth = 0.05)
    return value * (1 + growth)
end

println(project_one_year(100))
println(project_one_year(100; growth = 0.10))
```

Call the grow function

- Use grow to grow 100 by 5 percent and print the result.
- Use grow to grow 1000 by 8 percent and print the result.
- Use grow with your own value and your own growth rate.

Write two new functions

Write this function and test it with your own numbers:

```
function trade_balance(exports, imports)
    return exports - imports
end
```

Then write this function and test it with your own numbers:

```
function resource_balance(gdp, exports, imports)
    return gdp + exports - imports
end
```

Use keyword arguments

- Call project_one_year(100) and print the result.
- Call project_one_year(100; growth = 0.10) and print the result.
- Call project_one_year with your own value and growth rate.

6. Loops

Create a new file called 06_loops.jl. Loops repeat the same code over several values.

Start from this code

```
sectors = [:agriculture, :industry, :services]

gdp = Dict{
    :agriculture => 100.0,
    :industry => 300.0,
    :services => 600.0,
}

for sector in sectors
    println(sector)
end
```

Print sector values

Add this loop below the starter code:

```
for sector in sectors
    println(sector, ": ", gdp[sector])
end
```

- Run the file.
- Change one GDP value and run the file again.

Calculate a total using a loop

Add this code:

```
total_gdp = 0.0

for sector in sectors
    total_gdp = total_gdp + gdp[sector]
end

println("Total GDP: ", total_gdp)
```

- Run the file and check the printed total.
- Add a fourth sector to sectors and gdp, then update the loop if needed.

Loop over years and sectors

Add this nested loop:

```
years = 2024:2026

for year in years
    for sector in sectors
        println(year, " ", sector)
    end
end
```

- Change the years range to 2024:2029 and rerun the file.
- Create an exports dictionary and use a loop to calculate total_exports.

7. SquareModels

Create a new file called 07_squaremodels.jl in the instructor-provided project folder. Run this section only in the prepared environment where JuMP and SquareModels are installed.

Start from this code

```
using JuMP
using SquareModels

years = 2025:2027

db = ModelDictionary(Model())

JuMP.@variables db.model begin
    GDP[years]
    C[years]
    I[years]
    X[years]
    M[years]
    import_share[years]
end

model_block = @block db begin
    GDP_identity[t = years],
        GDP[t] == C[t] + I[t] + X[t] - M[t]

    Import_equation[t = years],
        M[t] == import_share[t] * GDP[t]
end
```

```
println(model_block)
```

Run the basic block

- Run the starter code in the prepared project folder.
- If Julia cannot find JuMP or SquareModels, stop and ask for help before editing the file.

Add government consumption

- Add `G[years]` inside the `JuMP.@variables` block.
- Change the GDP identity to this:

```
GDP[t] == C[t] + I[t] + G[t] + X[t] - M[t]
```

- Run the file again.

Rename and add equations

- Rename `Import_equation` to `Imports` and run the file again.
- Add this comment directly above the `@block`:

```
# In a model block, == writes an equation. A single = assigns a value or defines an index.
```

- Add a new equation called `Export_equation` where `X[t]` equals 0.40 times `GDP[t]`.

```
Export_equation[t = years],  
    X[t] == 0.40 * GDP[t]
```

- Run the file again.
-